

GenAI Code Comparison & Analysis: C++ CEO Email Priority Queue

Author: Kaleb Blakemore

KUID: 3228599

Course: EECS 348 Software Engineering I

Date: September 17, 2026

GenAI Programs Used

OpenAI ChatGPT, accessed through the web browser chat interface.

Google Gemini, accessed through the web browser chat interface.

How I Accessed Them

I used both ChatGPT and Google Gemini through their web-based chat interfaces on my Mac. I started a new conversation with each one so that neither program had previous information about my assignment or previous code. This gave both programs the same starting point when generating their code.

Shared Prompt Used

I gave both GenAI programs the exact same prompt so that I could compare the code they produced fairly.

Prompt:

"Write a C++ program that implements a custom Maxheap-based priority queue from scratch (without using pre-existing heap libraries like `std::priority_queue`). The program must manage a CEO's incoming emails based on sender category hierarchy and recency. The sender categories and priorities are: Boss (highest), Subordinate, Peer, Important Person, and Other Person (lowest). If categories match, newer dates (format MM-DD-YYYY) must be prioritized over older dates. It should parse standard commands from standard input: EMAIL, NEXT, READ, and COUNT."

Generated Code

ChatGPT Generated Code: Baseline

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
```

```

using namespace std;

struct Email {
    string category, subject, date;
};

class MaxHeap {
private:
    vector<Email> heap;

    int getPriority(string cat) {
        if (cat == "Boss") return 1;
        if (cat == "Subordinate") return 2;
        if (cat == "Peer") return 3;
        if (cat == "Important Person") return 4;
        return 5;
    }

    bool compare(Email a, Email b) {
        if (getPriority(a.category) != getPriority(b.category))
            return getPriority(a.category) < getPriority(b.category);
        return a.date > b.date;
    }

    void siftUp(int i) {
        while (i > 0 && compare(heap[i], heap[(i - 1) / 2])) {
            swap(heap[i], heap[(i - 1) / 2]);
            i = (i - 1) / 2;
        }
    }

    void siftDown(int i) {
        int maxIndex = i;
        int l = 2 * i + 1, r = 2 * i + 2;
        if (l < heap.size() && compare(heap[l], heap[maxIndex])) maxIndex = l;
        if (r < heap.size() && compare(heap[r], heap[maxIndex])) maxIndex = r;
        if (i != maxIndex) {
            swap(heap[i], heap[maxIndex]);
            siftDown(maxIndex);
        }
    }

public:
    void insert(Email e) {
        heap.push_back(e);
        siftUp(heap.size() - 1);
    }

```

```

Email peek() { return heap.front(); }
void pop() {
    heap[0] = heap.back();
    heap.pop_back();
    if (!heap.empty()) siftDown(0);
}
int size() { return heap.size(); }
bool isEmpty() { return heap.empty(); }
};

```

Google Gemini Generated Code — Baseline

```

#include <iostream>
#include <vector>
#include <string>
#include <sstream>

using namespace std;

struct EmailEmailNode {
    string senderCategory;
    string subjectLine;
    string dateStr;
};

class PriorityQueueHeap {
private:
    vector<EmailEmailNode> dataList;

    int rankCategory(const string& cat) {
        if (cat == "Boss") return 1;
        else if (cat == "Subordinate") return 2;
        else if (cat == "Peer") return 3;
        else if (cat == "Important Person") return 4;
        return 5;
    }

    bool isHigherPriority(const EmailEmailNode& x, const EmailEmailNode& y) {
        int rx = rankCategory(x.senderCategory);
        int ry = rankCategory(y.senderCategory);
        if (rx != ry) return rx < ry;
        return x.dateStr > y.dateStr;
    }

public:
    void push(const EmailEmailNode& item) {
        dataList.push_back(item);
    }
};

```

```

        int curr = dataList.size() - 1;
        while (curr > 0) {
            int parent = (curr - 1) / 2;
            if (isHigherPriority(dataList[curr], dataList[parent])) {
                swap(dataList[curr], dataList[parent]);
                curr = parent;
            } else {
                break;
            }
        }
    }

    EmailEmailNode top() { return dataList.front(); }

    void pop() {
        if (dataList.empty()) return;
        dataList[0] = dataList.back();
        dataList.pop_back();
        int curr = 0;
        while (2 * curr + 1 < dataList.size()) {
            int left = 2 * curr + 1;
            int right = 2 * curr + 2;
            int target = left;
            if (right < dataList.size() && isHigherPriority(dataList[right],
dataList[left])) {
                target = right;
            }
            if (isHigherPriority(dataList[target], dataList[curr])) {
                swap(dataList[curr], dataList[target]);
                curr = target;
            } else {
                break;
            }
        }
    }

    bool empty() { return dataList.empty(); }
    size_t size() { return dataList.size(); }
};

```

Code Analysis

Correctness

Both programs understood the main idea of the assignment. They both created a heap using a vector, compared the sender categories, and used the date to decide which email should come first when the categories were the same.

One issue I noticed in both programs was the spelling of the category Important Person. The assignment uses *ImportantPerson* without a space, so I needed to make sure the final program matched the exact input the grader would use.

The ChatGPT code also gave me the basic heap operations I needed: inserting an email, looking at the highest-priority email, removing it, checking the size, and checking if the heap was empty.

The Gemini code had the main heap operations too, but it did not include the full command-processing portion of the program. Because the assignment requires the program to process EMAIL, NEXT, READ, and COUNT, I would have had to add more code before it could be used as the complete assignment.

Execution Time

Both programs use a standard heap structure, so their main heap operations have the same basic time complexity.

Inserting an email takes $O(\log N)$ because the new email may need to move up the heap.

Removing the highest-priority email takes $O(\log N)$ because the replacement email may need to move down the heap.

Looking at the highest-priority email takes $O(1)$ because it is stored at the front of the heap.

Counting the emails takes $O(1)$ because the vector keeps track of its size.

The two programs are therefore very similar when it comes to execution time. I did not see a major performance difference that would make one clearly better than the other.

Space Complexity

Both programs use a vector to store the emails in the heap. This means that if there are N emails, the heap uses $O(N)$ space.

Neither program creates another large data structure to hold duplicate copies of all the emails, so they are both reasonable in terms of memory usage.

Maintainability

For maintainability, I preferred the ChatGPT version. I found it easier to follow because the heap operations were separated into smaller helper functions such as `siftUp()` and `siftDown()`.

The Gemini version also works with the basic heap idea, but some of its naming was harder to follow. For example, `EmailEmailNode` and `PriorityQueueHeap` were more complicated names than I felt were necessary.

I also liked that the ChatGPT version called the heap simply `MaxHeap`, which matches the terminology used in the assignment.

Selection and Justification

I selected the ChatGPT program as my baseline.

The main reason I chose it was that I thought it gave me a better starting point for the assignment. The code was separated into smaller functions, which made it easier for me to understand what each part of the heap was doing. I could work on the heap itself separately from the input and command processing.

I also felt that the ChatGPT version would be easier for me to modify and debug. Since I was going to have to make changes to the generated code anyway, having a simpler structure made more sense for me.

Code Improvements Made

After choosing the ChatGPT program, I made several changes to turn it into my final program.

Corrected the Email Category Priority

Corrected the Email Category Priority

I changed the category names so that they match the assignment exactly. In particular, the assignment uses `ImportantPerson`, not `Important Person`.

I also changed the priority values so that the highest-priority category has the highest number. This makes the comparison work more naturally with a Maxheap.

The final priority order is:

`Boss` → `Subordinate` → `Peer` → `ImportantPerson` → `OtherPerson`

Added the Complete Command Processing

The original ChatGPT code only provided the heap itself. I added the main input loop so the program can read the test file one line at a time.

The final program can process:

EMAIL

NEXT

READ

COUNT

I also added checks for an empty heap so that NEXT and READ do not cause errors when there are no emails.

Added Email Parsing

I added code to separate the sender category, subject, and date from an EMAIL command.

I used the first comma to separate the category from the subject and the last comma to separate the subject from the date. This allows the subject to contain spaces, as required by the assignment.

I also added whitespace trimming so that extra spaces around the input fields do not become part of the stored email information.

Improved Comments and Documentation

I added a full prologue at the beginning of the program with my name, KUID, program description, inputs, outputs, creation date, revision date, collaborators, and sources.

I also added comments throughout the program explaining what the different sections and lines are doing. Since this assignment's rubric specifically asks for detailed comments, I wanted the comments to make it clear how the heap and command processing work.

I also identified the parts that were originally generated by ChatGPT and the changes that I made myself.

Result

Overall, I used the ChatGPT program as a starting point rather than submitting the generated code exactly as it was. I reviewed both programs, compared their correctness, execution time, space complexity, and maintainability, and then selected the one that gave me the structure I felt I could work with more easily.

The final version includes the MaxHeap, email priority system, date comparison, input parsing, and all of the required commands. I also tested and debugged the program so that it follows the required input and output format.